Tài khoản đăng nhập và phân quyền

Tài liệu dành cho người phụ trách đăng nhập, phiên làm việc, vai trò và quyền truy cập dữ liệu.

Hệ thống dùng tài khoản có sẵn trong bảng NGUOI_DUNG. Spring Security xác thực email và mật khẩu, lưu phiên đăng nhập, rồi áp dụng vai trò KHACH_HANG, NHAN_VIEN hoặc QUAN_TRI cho các API. Một số nghiệp vụ còn kiểm tra quyền trong service; khách vãng lai dùng phiên riêng để xem đơn của mình.

Quy ước đường dẫn: UI/... nghĩa là src/main/resources/UI/...; các lớp Java nằm dưới src/main/java/com/cinesnack/. Đoạn code trích dẫn ghi đường dẫn đầy đủ và số dòng, giúp tìm đúng file trong dự án.

1 Bản đồ luồng

1. **Mở trang:** authentication.html → api.js + authentication.js — HTML tạo form email/mật khẩu; script gắn sự kiện submit
2. **Lấy CSRF:** api.js → GET /api/csrf → CsrfController — trả token cho các yêu cầu POST
3. **Đăng nhập:** authentication.js → api.js → POST /api/auth/login — gửi form username=email, password
4. **Xác thực:** ApiSecurityConfig → UserRepository → NGUOI_DUNG — đọc hash mật khẩu, vai trò và trạng thái rồi để Spring Security so khớp
5. **Lấy hồ sơ phiên:** api.js → GET /api/auth/me → UserController → UserService → UserRepository — trả id, email, role cho giao diện
6. **Điều hướng:** authentication.js/admin.js — hiện trang phù hợp với role và ẩn nút không dùng
7. **Chặn API:** ApiSecurityConfig → controller → service — Spring chặn route trước; service còn kiểm tra người xem có sở hữu đơn hoặc là nhân viên
8. **Khách vãng lai:** GuestOrderController → GuestOrderAccess → HttpSession — lưu mã đơn trong phiên khách để tra cứu; không tạo tài khoản
9. **Đăng xuất:** authentication.js hoặc admin.js → api.js → POST /api/auth/logout — Spring kết thúc phiên, giao diện trở về trang chủ

2 Vai trò từng file

File	Việc file thực hiện và hàm cần đọc
UI/html/authentication.html	Form email/mật khẩu; nạp api.js và authentication.js. Ghi rõ chưa cung cấp đăng ký mới.
UI/assets/js/authentication.js	Gắn submit, gọi dangNhap(), điều hướng theo role; tạo nút đăng nhập/đăng xuất chung.
UI/assets/js/api.js	layCsrf(), goi(), taiKhoan(), dangNhap(), dangXuat(); giữ cookie phiên bằng same-origin.
CsrfController.java	GET /api/csrf trả CsrfToken của Spring Security.
ApiSecurityConfig.java	Định nghĩa PasswordEncoder BCrypt, UserDetailsService, route công khai và route theo vai trò.
UserRepository.java	SELECT NGUOI_DUNG theo email bằng tham số ?; ánh xạ hash/role/status thành User.
User.java	record id/email/passwordHash/role/status; laNhanVien() đúng cho

	NHAN_VIEN hoặc QUAN_TRI.
UserService.java	layTaiKhoanHoatDong() kiểm tra trạng thái; kiemTraQuyenTruyCapDon() kiểm tra quyền xem đơn.
UserController.java	GET /api/auth/me trả id, email và role; không trả passwordHash.
GuestOrderAccess.java	Ghi và đọc tối đa 100 mã đơn khách vãng lai trong HttpSession.
GuestOrderController.java	Tạo/xem/hủy đơn khách dựa vào các mã đơn thuộc phiên.
OrderService.java	taoDonTaiQuay() chỉ nhận NHAN_VIEN; layDonDuocPhepTruyCap() dùng UserService.
PaymentService.java	Kiểm tra nhân viên trước khi xác nhận tiền mặt hoặc chuyển khoản.
UI/assets/js/admin.js	Ẩn phần quản trị với staff, hiện nút tạo đơn tại quầy cho staff; đây là lớp hiển thị.
database/cinesnack.sql	Bảng NGUOI_DUNG có email duy nhất, mat_khau_hash, vai_tro, trang_thai; chứa tài khoản mẫu.

3 Đọc các đoạn lệnh quan trọng

Mã nguồn: src/main/resources/UI/assets/js/api.js · dòng 4–11

```
4    async function layCsrf() {
5      const ph = await fetch("/api/csrf", {
6        credentials: "same-origin",
7        headers: { Accept: "application/json" },
8      });
9      if (!ph.ok) throw new Error("Không thể lấy token phiên. Vui lòng thử lại.");
10     csrf = await ph.json();
11     return csrf;
```

Giải thích: fetch lấy CSRF token từ máy chủ; credentials: same-origin gửi cookie phiên cùng tên miền. Token được giữ tạm trong biến csrf để dùng cho POST.

Mã nguồn: src/main/resources/UI/assets/js/api.js · dòng 13–27

```
13   async function goi(url, { method = "GET", body, form = false } = {}) {
14     if (!/^\/api\//.test(url)) throw new Error("Đường dẫn API không hợp lệ.");
15     const headers = { Accept: "application/json" };
16     if (!["GET", "HEAD", "OPTIONS"].includes(method)) {
17       const token = csrf || (await layCsrf());
18       headers[token.headerName] = token.token;
19     }
20     if (body !== undefined)
21       headers["Content-Type"] = form ? "application/x-www-form-urlencoded" : "application/json";
22     let ph;
23     try {
24       ph = await fetch(url, {
25         method,
26         headers,
27         credentials: "same-origin",
```

Giải thích: goi() chỉ cho URL bắt đầu /api/. Với phương thức thay đổi dữ liệu, code thêm CSRF header. body form được mã hóa bằng URLSearchParams, JSON được mã hóa bằng JSON.stringify.

Mã nguồn: src/main/resources/UI/assets/js/api.js · dòng 59–67

```
59   async function dangNhap(email, password) {
60     await goi("/api/auth/login", {
61       method: "POST",
62       form: true,
63       body: { username: email, password },
64     });
65     csrf = undefined;
66     await layCsrf();
```

```
67      return taiKhoan();
```

Giải thích: Đăng nhập gửi username và password theo tên trường Spring Security. Sau thành công, xóa token cũ và lấy token mới vì phiên có thể đã đổi; cuối cùng gọi /api/auth/me.

Mã nguồn: src/main/java/com/cinesnack/config/ApiSecurityConfig.java · dòng 18–42

```
18      @Bean
19      public PasswordEncoder boMaHoaMatKhau() {
20          return new BCryptPasswordEncoder(12);
21      }
22
23      @Bean
24      public UserDetailsService dichVuChiTietNguoiDung(UserRepository khoNguoiDung) {
25          return thuDienTu -> {
26              var taiKhoan = khoNguoiDung.timTheoThuDienTu(thuDienTu);
27              if (
28                  taiKhoan == null ||
29                  !Set.of("KHACH_HANG", "NHAN_VIEN", "QUAN_TRI").contains(taiKhoan.vaiTro())
30              ) {
31                  throw new UsernameNotFoundException("Tài khoản không hợp lệ");
32              }
33              return User.withUsername(taiKhoan.thuDienTu())
34                  .password(taiKhoan.matKhauHash())
35                  .roles(taiKhoan.vaiTro())
36                  .disabled(!"HOAT_DONG".equals(taiKhoan.trangThai()))
37                  .build();
38          };
39      }
40
41      @Bean
42      public SecurityFilterChain baoMatApi(HttpSecurity baoMatHttp) throws Exception {
```

Giải thích: @Bean đăng ký BCryptPasswordEncoder(12) và UserDetailsService. Repository tìm email; chỉ ba vai trò hợp lệ được nạp; disabled(true) nếu trạng thái khác HOAT_DONG. Spring Security so mật khẩu với hash.

Mã nguồn: src/main/java/com/cinesnack/config/ApiSecurityConfig.java · dòng 66–85

```
66                          .hasRole("QUAN_TRI")
67                          .requestMatchers("/api/guest/orders", "/api/guest/orders/**")
68                          .permitAll()
69                          .requestMatchers(HttpMethod.POST, "/api/admin/orders")
70                          .hasRole("NHAN_VIEN")
71                          .requestMatchers("/api/admin/**")
72                          .hasAnyRole("NHAN_VIEN", "QUAN_TRI")
73                          .requestMatchers(HttpMethod.POST, "/api/orders/*/payments/cash")
74                          .hasAnyRole("NHAN_VIEN", "QUAN_TRI")
75                          .requestMatchers(HttpMethod.POST, "/api/orders/*/payments/transfer")
76                          .hasAnyRole("NHAN_VIEN", "QUAN_TRI")
77                          .anyRequest()
78                          .authenticated()
79                  )
80                  .formLogin(dangNhap ->
81                      dangNhap
82                          .loginProcessingUrl("/api/auth/login")
83                          .successHandler((req, res, auth) -> res.setStatus(204))
84                          .failureHandler((req, res, error) -> res.sendError(401))
85                  )
```

Giải thích: requestMatchers ghép đường dẫn/phương thức với quy tắc. POST /api/admin/orders đòi ROLE_NHAN_VIEN; /api/admin/** còn lại nhận staff hoặc admin; xác nhận thanh toán nhận cả hai vai trò.

Mã nguồn: src/main/java/com/cinesnack/repository/UserRepository.java · dòng 18–34

```
18              """
19              SELECT nguoi_dung_id, email, mat_khau_hash, vai_tro, trang_thai
20              FROM NGUOI_DUNG WHERE email = ?
21              """,
22              (dong, viTri) ->
23                  new User(
24                      dong.getLong("nguoi_dung_id"),
25                      dong.getString("email"),
26                      dong.getString("mat_khau_hash"),
```

```
                    dong.getString("vai_tro"),
                    dong.getString("trang_thai")
                ),
            thuDienTu
        );
        return dsNguoiDung.isEmpty() ? null : dsNguoiDung.get(0);
    }
}
```

Giải thích: SELECT ... WHERE email = ? là truy vấn tham số: email không được ghép vào SQL. RowMapper tạo User từ một dòng; không tìm thấy thì trả null.

Mã nguồn: src/main/java/com/cinesnack/controller/UserController.java · dòng 17–21

```
    @GetMapping("/api/auth/me")
    public Map<String, Object> layTaiKhoan(Principal principal) {
        var u = nguoiDung.layTaiKhoanHoatDong(principal.getName());
        return Map.of("id", u.maSo(), "email", u.thuDienTu(), "role", u.vaiTro());
    }
```

Giải thích: Principal đại diện người đã đăng nhập; getName() là email. API /me chỉ trả id, email, role; không đưa mật khẩu hash về trình duyệt.

Mã nguồn: src/main/java/com/cinesnack/service/UserService.java · dòng 27–39

```
    public void kiemTraQuyenTruyCapDon(User nguoiDung, Order donHang) {
        if (donHang == null) throw new ResponseStatusException(
            HttpStatus.NOT_FOUND,
            "Không tìm thấy đơn hàng"
        );
        if (
            !nguoiDung.laNhanVien() && !Long.valueOf(nguoiDung.maSo()).equals(donHang.maNguoiDung())
        ) {
            throw new ResponseStatusException(
                HttpStatus.FORBIDDEN,
                "Bạn không có quyền truy cập đơn hàng này"
            );
        }
```

Giải thích: Nếu đơn không tồn tại thì 404. Người không phải nhân viên chỉ được xem đơn có nguoi_dung_id bằng id của mình; sai thì 403.

Mã nguồn: src/main/resources/UI/assets/js/authentication.js · dòng 89–96

```
  function destination(account) {
    const worker = ["NHAN_VIEN", "QUAN_TRI"].includes(account.role);
    const fallback = worker ? "admin.html" : "orders.html";
    if (!url) return fallback;
    const allowed =
      (account.role === "NHAN_VIEN" && /^\/html\/(admin|booking|checkout)\.html$/.test(url.pathname))
||
      (account.role === "QUAN_TRI" && url.pathname === "/html/admin.html") ||
      (!worker && /^\/html\/(booking|checkout|orders|movies|index)\.html$/.test(url.pathname));
```

Giải thích: destination() chọn trang đích theo vai trò và chỉ chấp nhận URL nội bộ nằm trong danh sách cho phép. Điều này tránh chuyển hướng tùy ý sau đăng nhập.

Mã nguồn: src/main/java/com/cinesnack/security/GuestOrderAccess.java · dòng 18–37

```
        }
        synchronized (session) {
            @SuppressWarnings("unchecked")
            Set<String> maDon = (Set<String>) session.getAttribute(SESSION_KEY);
            if (maDon == null) {
                maDon = new LinkedHashSet<>();
                session.setAttribute(SESSION_KEY, maDon);
            }
            maDon.add(donHang.ma());
            while (maDon.size() > MAX_GUEST_ORDERS_PER_SESSION) {
                maDon.remove(maDon.iterator().next());
            }
        }
    }

```

```
33      public Set<String> layMaDon(HttpSession session) {
34          if (session == null) return Set.of();
35          synchronized (session) {
36              @SuppressWarnings("unchecked")
37              Set<String> maDon = (Set<String>) session.getAttribute(SESSION_KEY);
```

Giải thích: Mã đơn khách được lưu trong HttpSession, có synchronized để tránh cập nhật đồng thời và giới hạn 100 mã. Khách chỉ truy cập các mã thuộc phiên đang dùng.

4 Ma trận quyền theo code hiện tại

Hành động	Khách	Nhân viên	Quản trị
Xem phim, suất, bắp nước	Có	Có	Có
Tạo đơn tại quầy	Không	Có	Không
Xem đơn toàn rạp	Không	Có	Có
Xác nhận tiền	Không	Có	Có
Quản lý phim và suất	Không	Không	Có
Xem đơn khách trong phiên	Có	Có nếu có phiên	Có nếu có phiên

Lưu ý: cột “khách” ở dòng cuối nói về khách vãng lai với đúng mã đơn nằm trong phiên. Tài khoản KHACH_HANG xem đơn gắn với id của mình qua API /api/orders. Các nút bị ẩn bởi JavaScript không quyết định quyền; kiểm tra có hiệu lực nằm ở Spring Security và service.

5 Trường hợp cần nhớ

- **Đăng ký:** Giao diện có trường đăng ký cũ bị ẩn/disabled, nhưng ghi rõ chưa cung cấp đăng ký; không có API đăng ký trong luồng hiện tại.
- **Tài khoản ngừng hoạt động:** Spring Security disabled tài khoản; UserService cũng từ chối khi trạng thái khác HOAT_DONG.
- **Mật khẩu:** Cơ sở dữ liệu lưu mat_khau_hash; trình duyệt gửi mật khẩu khi đăng nhập nhưng /api/auth/me không trả hash.
- **Phân quyền hai lớp:** ApiSecurityConfig chặn route; service kiểm tra tiếp quyền truy cập đúng đơn hoặc đúng vai trò nghiệp vụ.
- **Staff và admin:** Staff mới tạo đơn tại quầy. Cả hai được xem đơn toàn rạp và xác nhận thanh toán theo mã hiện tại.
- **CSRF:** POST phải gửi token; thiếu token có thể nhận 403 dù tài khoản đã đăng nhập.

6 Ví dụ đăng nhập và quyền truy cập

Nhân viên nhập email và mật khẩu vào authentication.html. authentication.js gọi api.js.dangNhap(); api.js gửi POST /api/auth/login cùng CSRF token và cookie phiên. Spring Security hỏi UserRepository để lấy passwordHash, role, status; BCrypt kiểm tra mật khẩu. Sau HTTP 204, trình duyệt gọi /api/auth/me để lấy role NHAN_VIEN và đi tới admin.html. Tại đó nhân viên thấy nút tạo đơn tại quầy. Nếu cùng phiên gọi API quản lý phim, ApiSecurityConfig trả 403 vì route này đòi QUAN_TRI.

Một tài khoản KHACH_HANG gọi GET /api/orders/{code} của người khác có thể vượt qua bước 'đã đăng nhập' nhưng vẫn bị UserService.kiemTraQuyenTruyCapDon() từ chối vì userId khác. Đây là lý do không thể chỉ kiểm tra vai trò ở JavaScript hoặc chỉ kiểm tra đăng nhập ở controller.

7 Từ vựng kỹ thuật

- **BCrypt:** Thuật toán băm mật khẩu; hệ thống so mật khẩu nhập với hash lưu trong bảng.
- **HttpSession:** Phiên máy chủ gắn với cookie trình duyệt; dùng cho đăng nhập và mã đơn khách vãng lai.
- **Principal:** Đối tượng Spring đưa vào controller để biết ai đang đăng nhập.
- **401, 403, 404:** 401 chưa đăng nhập hợp lệ; 403 đã vào nhưng không đủ quyền; 404 không tìm thấy tài nguyên.
- **@Bean:** Khai báo thành phần để Spring tạo và đưa vào những nơi cần dùng.